Applied Machine Learning Project

Graph-Based Synthetic Identity Fraud Detection

---

1. Background & Business Context

Traditional fraud detection models rely heavily on tabular data (e.g., transaction amounts, credit scores) to flag suspicious behavior. However, they are increasingly failing against a modern threat: **Synthetic Identity Fraud**.

In this scheme, bad actors blend real and fake information (e.g., a real Social Security/National ID number with a fake name and address) to create "Frankenstein" identities. These identities behave like normal customers for months or years, slowly building up credit, before executing a massive "bust-out" (maxing out lines of credit and disappearing). Because their individual tabular metrics look normal, they bypass standard ML algorithms. To catch them, organizations must look at *relationships*—how identities share phone numbers, IP addresses, and physical addresses across the digital ecosystem.

2. Project Objective

Develop a **Graph-based Machine Learning System** capable of mapping customer data as a network to identify complex, hidden relationships indicative of synthetic identity fraud rings.

3. Technical Architecture & Requirements

The project requires moving beyond traditional tabular dataframes and thinking in terms of nodes and edges.

3.1 1. Network Graph Construction

- Transform standard customer dataset rows into a graphical structure using libraries like `NetworkX` or `PyTorch Geometric`.
- **Nodes:** Should represent unique entities (e.g., Customer, Phone Number, Email Address, Physical Address).
- **Edges:** Should represent the linkages (e.g., Customer A *owns* Phone Number X; Customer B *resides at* Address Y).

3.2 2. Feature Engineering & Graph Analytics

- Compute fundamental graph metrics that act as fraud indicators:
  - *Degree Centrality:* E.g., multiple seemingly unrelated customers sharing the same phone number or device ID.
  - *Connected Components:* Identifying dense clusters of shared attributes that do not map to typical household structures.

3.3 3. Machine Learning Model Training

- Train a predictive model to classify a "Customer" node as either *Legitimate* or *Synthetic/Fraudulent*.
- **Approach:** Use a Graph Neural Network (GNN) such as GraphSAGE or GCN (Graph Convolutional Network) that learns not just from the customer's attributes, but from the attributes of their "neighbors" in the network.

4. Expected Deliverables

1. A Python codebase demonstrating the end-to-end pipeline: data ingestion, graph construction, feature extraction, and model training.
2. Generation of a synthetic dataset (using tools like Faker or basic scripting) that simulates normal customers alongside hidden fraud rings with shared attributes.
3. Visualizations of the graph network (e.g., using PyVis, Gephi, or Matplotlib) highlighting the detected fraudulent clusters.
4. Model evaluation metrics (Precision, Recall, F1-Score) demonstrating why the Graph approach outperforms a baseline tabular model (like Random Forest) on this specific problem.

5. Evaluation Criteria

- **Graph Design:** Logical soundness in how the nodes, edges, and graph features are structured to represent real-world digital identities.
- **Model Performance:** Effectiveness of the algorithm in catching hidden fraud rings with a low false-positive rate.
- **Data Synthesis:** Quality and realism of the generated dataset used to train and test the model.
- **Documentation & Insights:** Clarity in explaining the mechanics of Graph Machine Learning to non-technical stakeholders.